A Natural-Language Geospatial Query and Site-Evaluation System on PostGIS

Dr. Vaishali Wadhe
Department of Artificial Intelligence & Data Science
K.J. Somaiya Institute of Technology
Mumbai, India
vwadhe@somaiya.edu

Riddhesh Chaudhary
Department of Artificial Intelligence & Data Science
K.J. Somaiya Institute of Technology
Mumbai, India
riddhesh.c@somaiya.edu

Arpit Balmiki
Department of Artificial Intelligence & Data Science
K.J. Somaiya Institute of Technology
Mumbai, India
arpit.b@somaiya.edu

Chirayu Vyas
Department of Artificial Intelligence & Data Science
K.J. Somaiya Institute of Technology
Mumbai, India
chirayu.vyas@somaiya.edu

***Abstract*—Geospatial analysis supports urban planning, retail strategy and infrastructure decisions, yet traditional GIS platforms demand expertise in spatial query languages and specialised software. Large language models (LLMs) can lower this barrier, but systems that let the model answer directly tend to hallucinate place names and coordinates, which makes their output unverifiable. This paper presents GeoQuery, a Felt-style layered web map driven by a natural-language to PostGIS query engine grounded in open data. A user asks a spatial question in English, for example "hospitals within 1 km of a metro station". An LLM writes a PostGIS query against a documented schema, the query is executed on real data, and the system returns a GeoJSON answer layer, a grounded explanation and the SQL itself, so every answer carries provenance. On top of the engine we build intelligent decision tools: a click-to-evaluate site report, a competition and catchment analysis, a cost-to-open and break-even estimator, and a side-by-side comparison of up to three candidate locations. The system runs on PostgreSQL 16 with PostGIS 3.4, serves vector tiles with Martin, and renders them with MapLibre GL. It is loaded with Mumbai data from OpenStreetMap, a 76,000-row house-price dataset and area-level crime data, and its ETL is designed to generalise to other cities. We describe the architecture, the decision models and their current limitations, and report a first benchmark of query accuracy on 39 spatial questions.**

***Index Terms*—Generative AI, GIS, Natural Language Processing, Text-to-SQL, Spatial SQL, PostGIS, Site Selection**

I. Introduction

Geospatial data is central to urban planning, transportation, environmental monitoring and retail analytics. Geographic Information Systems (GIS) have traditionally been the main tools for such analysis, but they require users to understand spatial data handling, query languages such as SQL, and specialised software. This creates a barrier for the many people who have location questions but no GIS training.

Web platforms such as Felt have made map creation, collaboration and visualization much easier [1]. Even so, users must still translate a real-world question into manual operations on layers and filters. At the same time, LLMs have made natural-language interfaces to structured data practical, and recent work applies them to spatial data [2], [3].

A naive way to build such an interface is to let the LLM answer the question itself. Our first proof of concept (POC) did this and exposed three weaknesses. The model invented area names and coordinates. Ranking used a fixed hand-weighted formula over about 40 areas with synthetic 1 to 10 metrics. Map markers were placed with random coordinate jitter, and the maps were Leaflet raster maps with no real spatial computation. None of the outputs could be checked. Fig. 1 shows a typical exchange: the assistant answers with familiar area names, but nothing in the answer is computed from data.

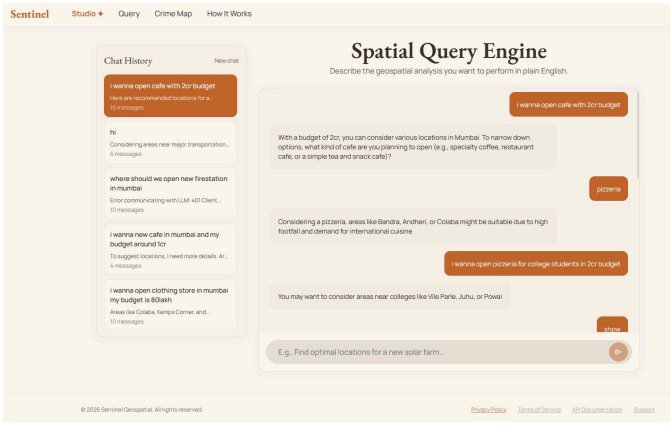


Fig. 1. The earlier chat interface. Area suggestions are produced by the LLM directly, with no spatial query behind them.

The central idea of this work is to replace "the LLM hallucinates the answer" with "the LLM writes a structured spatial query that is executed on real data". Every output is then verifiable and reproducible, and the generated SQL is shown to the user as provenance.

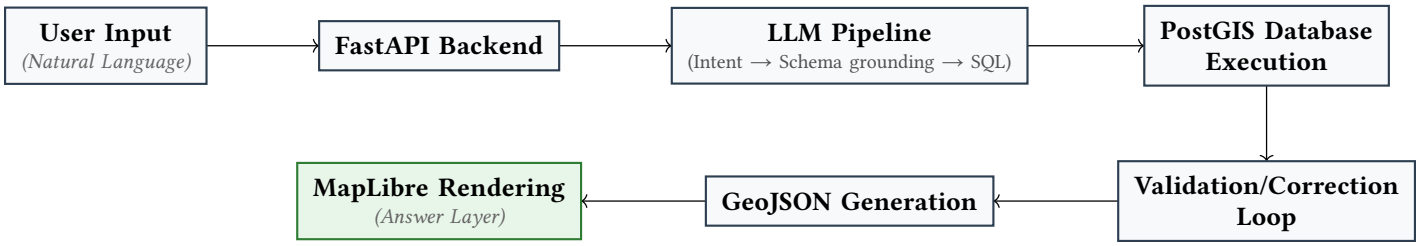


Fig. 2. System overview of GeoQuery: from a natural-language question to an answer layer on the map.

GeoQuery implements this idea as a full-screen layered map with a natural-language query bar. Beyond exploratory queries, it supports decision-oriented work for prospective business owners: evaluating a clicked location, analysing local competition and catchment demand, estimating the cost to open a store, and comparing candidate sites. The main contributions are:

1) A natural-language to PostGIS query engine that grounds the LLM in a documented schema and returns a GeoJSON layer, an explanation and the executed SQL.
2) A real-data foundation: an ETL pipeline that loads OpenStreetMap, real-estate listings and crime data into a single spatial database, designed to work for any city.
3) A set of decision tools (site evaluation, competition analysis, cost-to-open, location comparison) built on the same data.
4) An honest account of what is and is not yet validated, together with the evaluation plan for the missing pieces.

II. Literature Survey

Traditional GIS platforms such as ArcGIS and QGIS provide buffering, proximity analysis and spatial querying, but assume knowledge of spatial concepts, data formats and query languages. Cloud platforms such as Felt and Mapbox improve collaboration and visualization [1], yet users still drive them through layers, filters and map controls.

Natural-language interfaces to databases map a question to an executable query. Benchmarks such as Spider [4] and BIRD [5] have driven progress on general text-to-SQL, with LLM-based systems now dominating. Spatial questions add difficulty: the model must pick the right geometry types, apply spatial predicates and distance functions correctly, and respect coordinate reference systems. Early geospatial interfaces used rule-based mapping from keywords such as "near" and "within" to spatial operations. These are effective for simple queries but brittle for ambiguous or compositional ones.

More recent work uses LLMs for geospatial tasks. A multi-agent spatial text-to-SQL pipeline reports that adding an execution-based review stage improves accuracy by roughly 11 points [3]. ChatGeoAI converts natural language into executable geospatial operations for non-expert users [2]. GeoLLM shows that LLMs combined with OpenStreetMap features can model quantities for which ground truth is scarce [6]. We take the first two as the design reference for our query engine. The 11-point figure is a result from that work, not from ours. We treat GeoLLM as relevant to estimating metrics, such as rent, where no direct data exists.

Spatial decision support systems extend GIS with multi-criteria decision analysis (MCDA) for tasks such as site selection [7]. They rank candidates by combining normalised criteria with weights, but typically require expert configuration and expert interpretation of the result.

OpenStreetMap (OSM) provides free, globally available data on roads, amenities and land use [8], and tools such as OSMnx make it easy to acquire polygon-bounded extracts for any city [9]. PostGIS adds spatial types, GiST indexing and spatial functions to PostgreSQL, which makes it a suitable execution target for generated queries.

Existing systems tend to cover either visualization, query processing or decision support. We combine all three on one open-data stack, with the emphasis on verifiable query results.

III. Proposed System

GeoQuery lets users ask questions about geospatial data in plain English and then act on the results with decision tools. Its workspace is a full-screen map with a layer panel, a natural-language query bar, and panels for evaluating, comparing and costing locations.

Fig. 2 gives an overview. A question goes from the browser to the FastAPI backend, where an LLM pipeline checks its intent, grounds it in the schema and generates a PostGIS query. The query is validated and executed, a correction loop retries it on an error or an empty result, and the rows are returned as a GeoJSON answer layer drawn by MapLibre GL. The stages are detailed in the Methodology section.

A. Problem Statement and Objectives

a) *Problem Statement:*

Geospatial analysis is essential for business planning, but existing GIS tools demand knowledge of spatial data and query languages and mostly leave users to interpret patterns by hand. A person deciding where to open a cafe must weigh prices, transit, safety, competition and start-up cost, and no single easy tool combines these. LLM interfaces lower the barrier, but when the model supplies the answer directly, place names and coordinates can be invented and nothing can be verified. A system is needed that accepts natural language, computes answers on real spatial data, shows how each answer was obtained, and supports practical location decisions.

b) *Objectives:*

The primary objective is to build a natural-language geospatial query and decision-support system whose answers are computed from real data. The specific objectives are:

1) **To provide a natural-language interface** that lets users ask spatial questions in plain English without SQL or GIS knowledge.
2) **To translate questions into PostGIS queries** using an LLM grounded in a documented schema, and to return the result as a map layer, an explanation and the SQL.
3) **To build a spatial data foundation** on PostgreSQL and PostGIS from OpenStreetMap, real-estate listings and crime data, with an ETL pipeline that generalises to other cities.
4) **To serve and render layers efficiently** using vector tiles, with a toggleable layer panel and click-to-inspect interaction.
5) **To support location decisions** through a site evaluation report, competition and catchment analysis, a cost-to-open and break-even estimator, and a side-by-side comparison of candidate locations.
6) **To keep results explainable** by generating plain-English explanations and exposing the executed SQL.
7) **To remain cost-effective** by using open-source tools and open data.

B. Methodology

The system is a modular pipeline. A question enters through the query bar, the backend turns it into a spatial query, the database executes it, and the frontend draws the answer. Decision tools reuse the same database through dedicated endpoints.

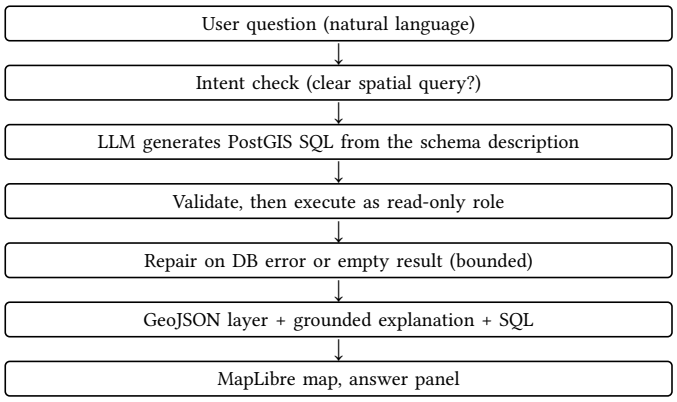


Fig. 3. Natural-language query pipeline of GeoQuery.

a) *Explanation of Methodology:*

1) **User input.** The user types a spatial question, either exploratory ("hospitals within 1 km of a metro station") or decision-oriented.
2) **Intent check.** A first LLM call decides whether the question is a clear spatial query and labels its task type (filter, proximity, ranking, aggregation, site selection or other). If the target is genuinely unknowable, for example "best place?", the system returns a short follow-up question instead of running a query.
3) **Schema grounding.** The SQL-generating call receives a compact description of the queryable schema in its prompt: the tables and columns of the admin, roads, transport, poi, realestate, demo, crime and env schemas with geometry types and the allowed values of key categorical columns, plus rules to filter every table by city_id, never reference an unlisted table or column, and emit one read-only SELECT with a geometry column named geom. Because the model can only reference data that exists, it cannot invent places. The same description is stored as a 34-row meta.schema_catalog table, which the engine does not read at query time.
4) **Query generation.** The LLM (Groq openai/gpt-oss-120b, behind a single client module) writes a PostGIS query using functions such as ST_DWithin and ST_Contains over EPSG:4326 geometry indexed with GiST. The text is validated before execution: it must be a single SELECT or WITH statement with no chained statements and no data-modifying or DDL keywords.
5) **Execution.** The query runs on PostgreSQL 16 with PostGIS 3.4 as a dedicated geo_readonly role, with a 15 second statement timeout and a cap of 5,000 returned features.
6) **Repair.** If the query fails validation or raises a database error, the error text is fed back to the model for a corrected query, with at most two such repairs. If a valid query returns zero rows, the model is asked once to loosen it. The best successful attempt is kept, preferring a non-empty one, and every attempt is returned to the client.
7) **Result generation.** Rows are converted to a GeoJSON answer layer, and the LLM writes a short explanation grounded in a sample of the returned rows. The generated SQL is returned alongside, so users can inspect how the answer was obtained.
8) **Visualization.** The answer layer is drawn on the map next to the base layers. Base layers are served as Mapbox Vector Tiles by Martin directly from PostGIS.

Chat history is stored in MongoDB. It is not used for spatial data.

C. Analysis, Framework and Algorithm

a) *System Framework Overview:* The system has three layers: a presentation layer (React and MapLibre GL), an application layer (FastAPI endpoints and the LLM client), and a data layer (PostGIS, Martin and MongoDB). This separation keeps the model, the query execution and the rendering independently replaceable.

b) *Algorithm for Query Processing:*

```
def answer(question, city_id):
    intent = llm.interpret(question)          # clear spatial
query?
    if not intent.is_clear:
        return intent.follow_up               # ask the user, no
```

```
query run
  best, error, empty = None, None, False
  for _ in range(MAX_REPAIRS + 2):
    sql = llm.generate_sql(question, SCHEMA_DOC, error,
empty)
    try:
      sql = validate_select(sql)        # single read-only
SELECT
      layer = execute_readonly(sql)     # geo_readonly, 15 s
timeout
    except Exception as e:
      error, empty = str(e), False      # feed the error back
      continue
    if best is None or layer.count > best.count:
      best = (sql, layer)
    if layer.count > 0: break
    if empty: break                     # one loosening attempt
only
    error, empty = None, True
  text = llm.explain(question, best.rows)   # grounded in
returned rows
  return best.layer, text, best.sql         # SQL returned as
provenance
```

The loop makes at most four generation attempts: the first, up to two repairs after errors, and one loosening after an empty result. This follows the execution-based review idea of [3] in a simplified form. The effect of the repair stage on accuracy has not yet been measured.

c) *Decision Support Models:* The decision tools are built on the same database and exposed as separate endpoints (/site-eval, /competitor-analysis, /business-estimate).

Site evaluation. For a clicked point and a radius between 0.5 and 3 km, the report gives the median price per sqft of nearby listings, a typical price band, a comparison with the city median, a breakdown by size band, the nearest metro, rail and bus stops, amenity counts (healthcare, education, food, banking, shopping), and a safety score with a risk level.

Competition and catchment analysis. The report gives the competitor count, density, nearest competitor, a saturation label and a 0 to 100 catchment demand index, and suggests lower-competition spots (green markers; competitors are red dots). Competitors are points in poi.place whose category matches the business type inside the chosen radius, and density is their count per km². The saturation label uses fixed bands (below 5 low, 5 to 15 moderate, 15 to 30 high, above 30 saturated). The demand index counts points of interest in five groups (healthcare, education, food, banking and shopping) and transit stops inside the radius, caps each count (40, 25, 40, 15, 30 and 12 respectively) so that dense areas do not dominate, and combines them with business-specific weights, for example a cafe weights education, shopping and transit. To suggest gaps, a 5 by 5 grid of candidate spots is laid over the radius, each scored as its demand index divided by one plus its competitors within 500 m. Up to three spots with a demand index of at least 15 and a score at least 15% higher than the evaluated spot are suggested. The bands and weights are heuristics set by judgement and have not been calibrated against business outcomes.

Cost-to-open and break-even. The estimator takes a store area in sqft and returns one-time setup costs, monthly running costs, staff cost and working capital. Because rental data is not available, monthly rent per sqft is estimated as the median sale price per sqft of listings within 2.5 km times 0.5% (a rent-to-value ratio) and a commercial premium of 1.15, with a city default of 120 rupees per sqft if no listing is nearby. The security deposit is six months of rent and working capital is three months of running costs. Other line items are produced by the LLM, with a heuristic fallback. Customers per day are the store area (per 100 sqft) times a benchmark rate for the business type, a footfall multiplier of 0.6 to 1.5 from amenities within 1 km and metro or rail access within 800 m, and a competition factor $\frac{1}{1+0.04n}$ floored at 0.6, where n is the number of competitors within 1 km. Revenue is customers times an average ticket (scaled by 0.8, 1.0 or 1.3 for economy, standard or premium tier) times 30 days; cost of goods is a fixed share of revenue, and profit is revenue minus cost of goods and the remaining monthly costs. The conservative, base and optimistic scenarios scale the customer count by 0.7, 1.0 and 1.3. Payback is the setup cost divided by monthly profit, and break-even revenue is the remaining monthly cost divided by one minus the cost-of-goods share. The per-type benchmarks are modelling assumptions set by the authors, not measured values, and because the LLM line items vary, two runs for the same site can give different totals, profit and payback.

Location comparison. Up to three pinned spots (A, B, C) are shown side by side across price, access, safety, competitors, rent, start-up cost, revenue, profit and payback. The best value in each row is marked.

Area-level weighted ranking (legacy dashboard). The original dashboard ranks areas with a standard weighted-sum MCDA model [7]. For area i with criteria m_{ik} (footfall, youth, access, rent and competition, each scored 0 to 10),

$$S_i = \sum_{k=1}^{K} w_k \, m'_{ik}, \quad \sum_{k=1}^{K} w_k = 1, \tag{1}$$

where $m'_{ik} = m_{ik}$ for benefit criteria (footfall, youth, access) and $m'_{ik} = 10 - m_{ik}$ where lower is better (rent and competition). The hand-picked weights are 0.30, 0.25, 0.20, 0.15 and 0.10 in that order, and the five highest-scoring areas are returned. In the POC the metrics were synthetic.

To test how much the ranking depends on these weights we ran a sensitivity study on the 40 scored areas (script in the eval/ folder). We compared the hand-picked weights with equal weights, rank-order centroid (ROC) weights, AHP weights (consistency ratio 0.011), entropy weights, CRITIC weights and three scenario weight sets (budget, student and premium), and we compared the weighted sum with TOPSIS under the baseline weights. Agreement with the baseline ranking was measured by Kendall's τ and by the overlap of the top five areas. We also perturbed each weight by up to 30% one at a time, and drew 1,000 random weight vectors from a uniform Dirichlet distribution (seed 42) to see how often each area stays in the top five.

The ranking is stable under small changes to a single weight: the worst-case τ for a 30% change in any one weight is 0.90 or higher (lowest for the access weight). It is not stable under a change of weighting scheme. AHP, ROC and the premium scenario keep the same top five as the baseline (Kendall τ of 0.92, 0.81 and 0.83), while equal weights, entropy weights, the student scenario, the budget scenario, TOPSIS and CRITIC share only 2, 2, 2, 1, 1 and 0 of the baseline's top five areas (τ of 0.41, 0.58, 0.87, −0.11, 0.36 and −0.01). Across random weight vectors the median τ against the baseline is 0.33 (5th to 95th percentile: −0.49 to 0.84), and no area stays in the top five in more than 45% of draws (Andheri West is highest at 44.5%). The weights therefore encode a preference, not a neutral measurement, which is one reason the GeoQuery workspace uses measured spatial queries in place of this fixed ranking. These results describe the 40-area scored dataset and should not be read as evidence about real-world site quality.

D. Details of Hardware and Software

a) *Hardware Requirements:* The system runs on a standard personal computer or laptop.

- Processor: Intel i5 or equivalent (or higher recommended)
- RAM: minimum 8 GB (16 GB recommended for large datasets)
- Storage: at least 20 to 50 GB free (OpenStreetMap data and database)
- Network: internet connection for the LLM API and geocoding

b) *Software Requirements:*

The stack is primarily open source.

Sr. No.	Software/Tool	Description
1	PostgreSQL 16 + PostGIS 3.4	Spatial database with GiST indexing; pgvector extension also installed
2	Martin	Serves Mapbox Vector Tiles directly from PostGIS
3	FastAPI + Uvicorn	REST backend for the query engine and decision tools
4	Groq API (openai/gpt-oss-120b)	LLM for SQL generation, explanations and cost line items
5	React 19 + Zustand	Frontend and client state
6	MapLibre GL	Vector-tile map rendering in the main workspace
7	Leaflet	Map in the legacy dashboard only
8	OSMnx	Polygon-based OpenStreetMap ingest for any city
9	Nominatim	Geocoding of 228 real-estate localities
10	MongoDB	Chat history only
11	Docker	Runs the database and supporting services

E. Design Details

a) *System Architecture Overview:*

TABLE I
System component stack.

Presentation	React 19, MapLibre GL, Zustand; layer panel, query bar, evaluation panel
Application	FastAPI endpoints, LLM client (groq_client.py), MongoDB chat history
Tiles	Martin serving MVT from PostGIS
Data	PostgreSQL 16 + PostGIS 3.4 + pgvector; schemas admin, roads, transport, poi, realestate, demo, crime, env; meta.schema_catalog; core.city
ETL	OSMnx ingest, Nominatim geocoding, crime data migration

b) *Module-wise Design:*

Frontend (Studio): The map is full-screen with OSM base tiles. The left panel toggles vector layers served by Martin, each with an opacity slider: administrative boundaries, road network, transit stops, businesses and POIs, real-estate listings, and a crime and safety score. Users can click any feature to inspect it, type a question into the query bar, or click a point to open the evaluation panel with its Competition and Cost-to-open tabs.

Backend API: FastAPI exposes /nlquery for natural-language queries, /site-eval, /competitor-analysis and /business-estimate for the decision tools, and /health. The original dashboard additionally uses /analyze, /generate, /explain, /query, /location-insight, /properties and /locations.

Spatial Database: The database holds all layers in EPSG:4326 with GiST indexes, and every row is tagged with a city_id so several cities can coexist. The schema catalog documents the tables, and the same description is embedded in the model prompt.

ETL: The ETL pipeline (etl/) ingests roads, transit, POIs and boundaries from OSM with polygon-based extraction, geocodes real-estate localities with Nominatim, and loads crime data. Because extraction is driven by a city polygon, the same pipeline can load other cities. The deployed instance is pre-loaded for Mumbai.

Legacy Dashboard: The /dashboard page is the original assistant view: a chat assistant that asks clarifying questions and ranks areas, a Leaflet map with a heatmap by factor, a Top Locations panel with explanations, and a location detail view. Where an area has no scored metrics, the detail view borrows them from the nearest scored area. We keep it as the baseline against which the GeoQuery workspace is compared.

c) *Design Considerations:*

- Verifiability: the SQL is always shown, and answers come from executed queries.
- Performance: vector tiles and spatial indexes keep map interaction fast.
- Generality: the city_id tag and polygon-based ETL support other cities.
- Cost efficiency: the stack uses open-source tools and open data.

F. Advantages And Limitation

a) *Advantages:*

1) **Verifiable answers.** Results come from executed queries on real data, and the SQL is shown, which addresses the hallucination problem of direct LLM answers.
2) **Natural-language interface.** Users need no SQL or GIS knowledge.
3) **Decision support.** Site evaluation, competition analysis, cost-to-open and comparison turn a map into a practical planning tool.
4) **Open and low cost.** The stack uses open-source software and open data.
5) **Generalisable.** Polygon-based ETL and city_id tagging allow other cities to be added.

b) *Limitations:*

1) **Small benchmark.** The NL to SQL benchmark has 39 questions and was run once, so its accuracy figures carry substantial uncertainty, and it covers only filter, proximity, area, ranking, listing and compositional questions.
2) **Repair stage not exercised.** The engine retries after a database error or an empty result, but no query on the benchmark needed a retry, so the benefit of repair is unmeasured.
3) **Basic SQL safeguards.** Generated SQL runs under a read-only role with a statement timeout and a row cap, and is checked to be a single SELECT. The check is keyword-based and not a full SQL parser, and there is no table allow-list beyond the role's grants.
4) **No real rental data.** Rent is estimated from sale prices, so cost-to-open figures are approximate.
5) **Heuristic decision models.** The saturation bands, demand weights and per-type revenue benchmarks are set by judgement and have not been calibrated against real outcomes.
6) **LLM-generated cost items.** The non-rent setup and running costs, and the summary paragraph of the cost report, are written by the LLM and are not reconciled with the computed totals. Totals, profit and payback can therefore differ between runs for the same site, and the summary text can disagree with the figures beside it.
7) **Coarse listing locations.** Real-estate listings are geocoded at the locality level (228 localities), not per property.
8) **Radius-based catchments.** Catchments are circles, not travel-time isochrones.
9) **Data coverage.** Only Mumbai is loaded, and OSM completeness varies. Crime data is area-level.
10) **Model variability.** The LLM is called without a fixed temperature, so the same question can yield different SQL.
11) **No saved sites or mobile layout.** Pinned locations are not persisted, and the interface is designed for desktop.

IV. Results And Discussion

A. Implemented System

The system is implemented end to end. Table II lists the Mumbai data currently loaded.

TABLE II
Mumbai data loaded in the current instance.

Dataset	Size
Administrative boundaries	48
Road segments	16.8k
Transit stops	2.2k
Businesses and POIs	10.8k
Real-estate listings (CSV)	76,000 rows
Geocoded real-estate localities	228
Schema catalog entries	34

Compared with the POC, the GeoQuery workspace replaces invented coordinates with coordinates from the database, replaces random marker jitter with real geometries, replaces raster Leaflet maps with vector tiles, and replaces fixed synthetic metrics with metrics computed from nearby listings, transit and amenities.

B. Qualitative Demonstration

A query such as "hospitals within 1 km of a metro station" is turned into a spatial SQL query, executed, and drawn as an answer layer, with the explanation and the SQL available in the panel.

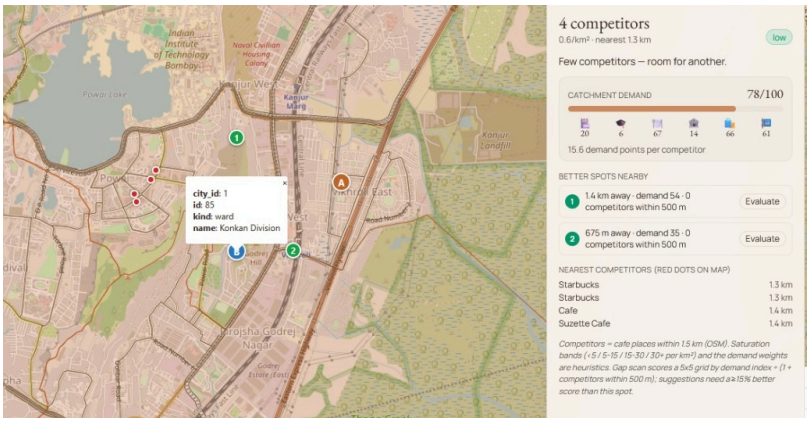
Clicking a location opens a site report with price, access, amenity and safety information, and the Competition and Cost-to-open tabs extend it to a business decision. Fig. 4 shows this for a cafe at Vikhroli (B), compared with Kanjurmarg (A). The competition tab finds four cafes within 1.5 km, the nearest 1.3 km away, so both sites have no competitor within 1 km. It also suggests two lower-competition spots (green). The comparison table puts measured site attributes (price, transit, amenities, safety) next to the modelled business figures.

C. Comparative Analysis

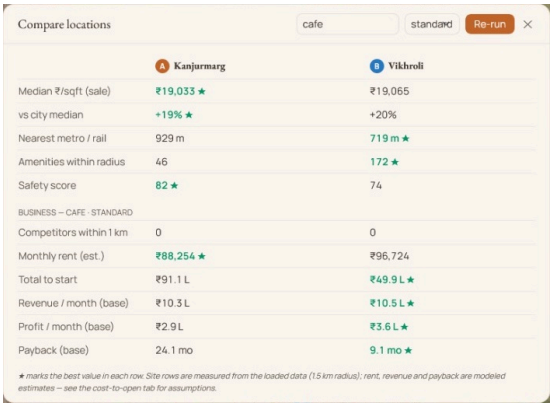
We compare the system along two axes: the query engine against its own ablations and against the direct-LLM approach of the POC, and the weighting schemes of the legacy area ranking.

a) *Query engine versus ablations and the direct-LLM baseline:*

The benchmark in eval/nl2sql/ contains 39 spatial questions in six categories (filters, proximity, named areas, rankings, listings and compositional queries), each with a hand-written gold SQL query. The expected answer is the executed result of the gold query, and a predicted layer is scored by comparing geometry sets (exact match and F1), so column names do not matter. Three configurations are compared: the full engine; the same engine restricted to its first generation attempt, obtained by re-executing the first SQL of each full run, which makes it a paired comparison; and the direct-LLM baseline, in which the model names places and



(a) Competition tab for a cafe at Vikhroli (B), 1.5 km radius: competitors in red, suggested spots in green.



(b) Comparison of Kanjurmarg (A) and Vikhroli (B), cafe, standard tier.

Fig. 4. Decision tools in the GeoQuery workspace. Site rows in (b) are measured within 1.5 km; rent, start-up cost, revenue, profit and payback are modelled estimates from a single run.

coordinates without a database. The baseline returns no SQL, so it is scored on grounding: the share of returned names that exist in the database and the share of returned points that lie near a real feature. Results are from a single run with openai/gpt-oss-120b and default sampling. The intent-check and explanation calls were skipped, because they do not change the SQL.

TABLE III
BENCHMARK RESULTS ON 39 QUESTIONS, SINGLE RUN.

Configuration	Exact	Geom. F1	Executed
Full engine	74.4%	88.4%	100%
First attempt only (no repair)	74.4%	88.4%	100%

The full engine returns exactly the gold layer for 29 of the 39 questions (74.4%, geometry F1 88.4%), and every generated query executed and returned rows (Table III, Fig. 5). The repair stage was never triggered: all 39 first attempts executed and returned rows, so the first-attempt-only configuration scores identically, and this benchmark shows no benefit from repair. Accuracy is highest for named-area and ranking questions (both 100%) and lowest for listings (25%). Eight of the ten failures come from a row cap and not from a wrong query. The engine's schema notes tell the model to use LIMIT 500, so questions whose gold answer is larger than 500 rows (hospitals, listings, hospitals near two kinds of station) are truncated, and joined queries without DISTINCT let duplicate rows use up the limit (for example the banks near bus stops and the schools in Bandra near bus stops). The other two failures are over-inclusive category choices: universities were matched together with colleges, and motorways with a pattern broader than the gold road class. We count all ten as failures, but they point to the prompt rules and to the exact-match scoring more than to a lack of spatial reasoning.

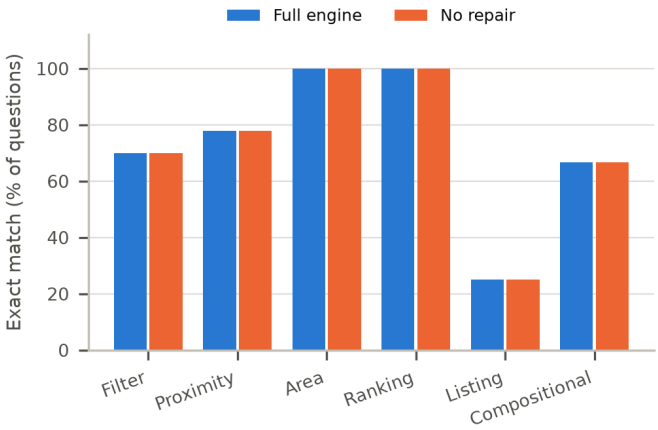


Fig. 5. Exact-match accuracy by question category. The first-attempt-only configuration is identical to the full engine because no repair was needed.

The direct-LLM baseline gives a very different picture (Fig. 6). Only 19.5% of the place names it returned exist in the database, 57.0% of its coordinates lie within 150 m of some real feature, and only 10.6% lie within 200 m of a feature in the gold answer. The coordinate figure is generous, because Mumbai is dense and a random point is often near some point of interest, so the other two figures are more informative. The baseline also returns no SQL, so none of its answers can be checked. The median latency of the engine was 4.5 s (one LLM call plus the database, without the intent-check and explanation stages that the shipped engine adds) and that of the baseline 13.2 s. With 39 questions and one run, differences of a few questions should not be over-interpreted. A further ablation, in which the schema description lists only table names, is implemented in the benchmark code but was not run.

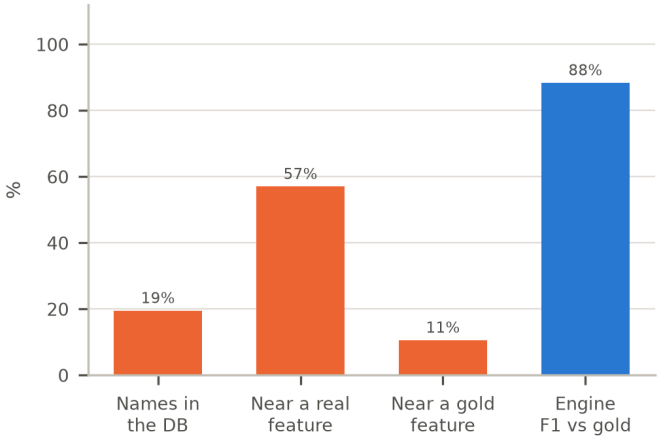


Fig. 6. Grounding of the direct-LLM baseline (three left bars) next to the engine's geometry F1 against the gold answer (right bar). The metrics differ: the baseline bars show how much of its output is real, the engine bar its agreement with the gold layer.

b) *Weighting schemes of the legacy ranking:*

The comparison of weighting schemes is described with the decision support models above. Fig. 7 shows how far each scheme's ranking is from the hand-picked one, and Fig. 8 shows how little the ranking changes when a single weight is changed by up to 30%.

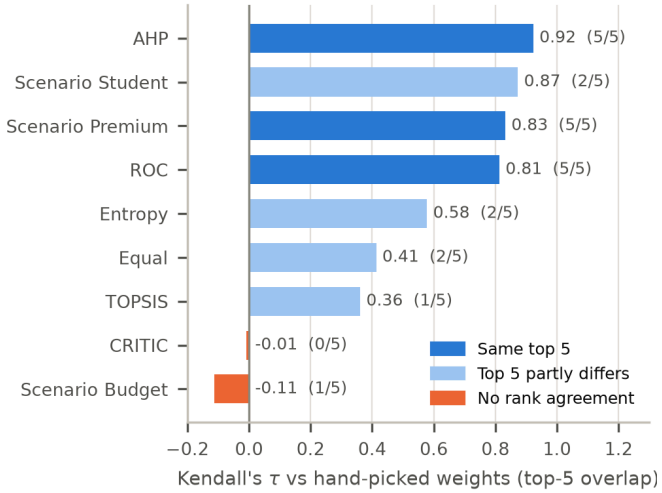


Fig. 7. Rank agreement (Kendall's τ) of each weighting scheme with the hand-picked weights, and the overlap of the top-five areas.

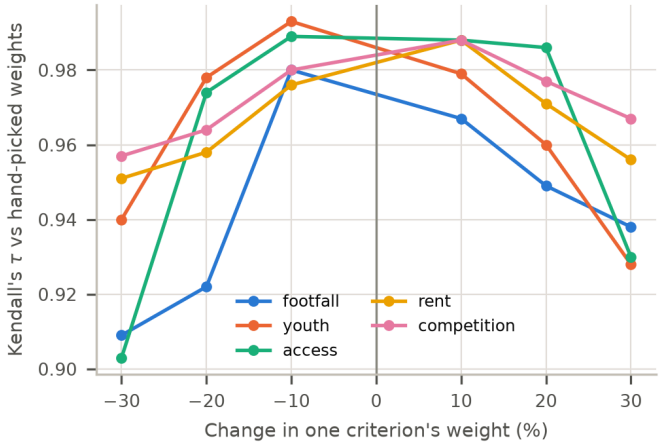


Fig. 8. One-at-a-time sensitivity: Kendall's τ against the hand-picked ranking when one criterion's weight is changed by up to 30%, the others being renormalised.

D. Evaluation Status

The table summarises what has and has not been measured.

Measurement	Status
NL to SQL execution accuracy on a labelled benchmark of spatial questions	39 questions, single run: 74.4% exact match, 88.4% geometry F1
Effect of the execution-based repair stage	Repair never triggered on the benchmark (0 of 39 questions), so no measurable effect
Latency of /nlquery and the decision endpoints	/nlquery generation step: median 4.5 s, 95th percentile 13.8 s; decision endpoints not measured
Weight sensitivity of area ranking	Done on the 40-area dataset, see the decision support section
Accuracy of rent estimate against real rents	No rental data yet

E. Discussion

Grounding the model in a documented schema and returning the executed SQL changes the nature of the interface: answers can be checked, reproduced and corrected. The decision tools show that the same data supports practical questions, such as comparing three candidate shop locations on price, access, safety and competition.

The main weakness of the current work is evaluation. The benchmark is small, was run once, and did not exercise the repair stage, so we cannot say how much repair helps or how the engine behaves on harder questions. The safeguards, a read-only role, a statement timeout, a row cap and a single-statement check, limit the damage a malformed query can do, but they have not been tested against adversarial input. Cost and revenue estimates inherit the approximations in the rent model, the per-type benchmarks and the LLM-produced line items, and should be treated as planning aids. Because

those line items are regenerated on each run, the same site can receive different start-up and payback figures, so the measured site attributes are more reliable than the modelled business figures. The weight study shows that the legacy area ranking is sensitive to the choice of weighting scheme.

V. Conclusion And Future Scope

A. Conclusion

GeoQuery shows that a natural-language interface to geospatial data can be built so that answers are computed on real data and are inspectable. By pairing a schema-grounded LLM with PostGIS, vector tiles and open data, it avoids the invented places and coordinates of direct LLM answers, and it extends the query engine with site evaluation, competition analysis, cost-to-open modelling and location comparison for practical decisions. The system is functional for Mumbai, and its ETL is designed to generalise. A first benchmark of 39 questions shows 74.4% exact-match accuracy against hand-written gold queries, with most failures caused by a row cap, whereas a direct-LLM baseline returns real place names only 19.5% of the time. A larger benchmark remains to be built.

B. Future Scope

1) **Query reliability:** extend the benchmark (more questions, repeated runs, other models and harder queries that exercise the repair stage) in the style of Spider and BIRD [3], [4], [5], and remove the row-cap and over-inclusion failures found here.
2) **Safety:** replace the keyword check with parser-based validation and a table allow-list.
3) **Data:** integrate real rental data, and extend the metrics modelling with LLM and OSM features where ground truth is scarce [6].
4) **Analysis:** replace radius catchments with isochrones, and calibrate the saturation bands and revenue benchmarks against real outcomes.
5) **Cost model:** fix the LLM line items per business type and tier (or cache them per site), and generate the report summary from the computed totals, so that repeated runs agree.
6) **Usability:** saved sites and a mobile layout.
7) **Coverage:** load additional cities using the existing ETL.

References

[1] Felt, "Felt AI: AI-powered GIS and mapping platform." [Online]. Available: https://help.felt.com/felt-ai

[2] A. Mansourian and R. Oucheikh, "ChatGeoAI: Enabling Geospatial Analysis for Public through Natural Language, with Large Language Models," *ISPRS International Journal of Geo-Information*, vol. 13, no. 10, p. 348, 2024.

[3] A. K. Kazazi, Z. Li, M. N. Lessani, and G. Cervone, "From Questions to Queries: An AI-powered Multi-Agent Framework for Spatial Text-to-SQL," *arXiv preprint arXiv:2510.21045*, 2025.

[4] T. Yu *et al.*, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 3911–3921.

[5] J. Li *et al.*, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] R. Manvi, S. Khanna, G. Mai, M. Burke, D. Lobell, and S. Ermon, "GeoLLM: Extracting Geospatial Knowledge from Large Language Models," in *International Conference on Learning Representations (ICLR)*, 2024.

[7] J. Malczewski, *GIS and Multicriteria Decision Analysis*. New York: John Wiley & Sons, 1999.

[8] M. Haklay and P. Weber, "OpenStreetMap: User-Generated Street Maps," *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008.

[9] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.